SECURITY TESTING / TECHNICAL REPORT

Coverage-Guided Fuzzing of libmobi v0.10

Honggfuzz campaigns, memory-safety analysis, crash minimization, patch validation and evidence audit



SCOPE	KEY FINDING	PROVENANCE
21 numbered campaigns	1 distinct historical font-decoding defect	17 log-derived final summaries
ASan / UBSan / Valgrind	3 confirmed crashing inputs; 1 root cause	Raw logs, crash files and regression evidence

REPORT DATE | 23 October 2026

STATUS | Evidence-bounded standalone workstream report. Missing logs are explicitly marked; group-wide fuzzer comparisons are out of scope.

Independent security-testing case study based on recorded experiments, crash diagnostics, regression checks and LLVM coverage evidence.

Executive summary

This study evaluated the coverage-guided fuzzer Honggfuzz against libmobi v0.10, an earlier release of a C library that processes MOBI e-book files. The experimental program comprised 21 numbered campaign entries, including smoke tests, longer fuzzing runs, sanitizer variants, corpus minimization, dictionary comparisons and diagnostic runs. Seventeen campaign final summaries were checked directly against the archived Honggfuzz logs and matched the exported TSV with no numerical discrepancies; four campaigns lack a parseable final summary.

The primary security result is one distinct, reproducible historical out-of-bounds read in mobi_decode_font_resource (src/util.c:2543). Independent crash inputs saved by HF-09, HF-13 and HF-19 converged on that defect. A one-byte mutation of a valid input reproduced the fault, while a structure-aware minimization reduced the 84,412-byte file to 15,531 bytes (81.60% smaller). A custom bounds-validation patch prevented the known failure across the tested regression inputs, and Valgrind independently confirmed an invalid one-byte read in the vulnerable binary.

The HF-13 corpus was reduced from 1,467 to 995 files (32.17%) and from 480,881,076 to 276,162,698 bytes (42.57%). Independently parsing the supplied original and minimized LCOV exports confirms identical covered source-identifier sets (5,921 lines, 2,395 branches, 276 functions; zero lost or gained identifiers). Both corresponding merged LLVM profiles are also present and parseable. Two repeated runs per dictionary condition yielded no clear, repeatable advantage for the 15-token MOBI dictionary.

KEY METHODOLOGICAL SAFEGUARD

Three crash-producing samples are not three unique bugs. Similarly, two HF-04 timeout-related signal reports are not counted as confirmed security vulnerabilities. A missing log or incorrectly configured experiment is retained transparently in the register rather than assigned reconstructed statistics.

Contents

01 Research question and scope
02 Target, environment and builds
03 Experimental design and measurement
04 Complete campaign register (HF-01–HF-21)
05 Core results and configuration comparison
06 Confirmed vulnerability and reproducer
07 Crash and corpus minimization
08 Sanitizer cross-validation and regression tests
09 Dictionary experiments
10 Upstream history, CVEs and OSS-Fuzz
11 Validity threats and methodological reflection
12 Conclusions and assignment hand-off
13 Appendix A: audit details
Appendix B: evidence manifest
Appendix C: primary evidence verification
Appendix D: independently verified seed and coverage evidence
References (following Appendix D)

1 Research question and scope

The research question is: How effectively does Honggfuzz exercise malformed MOBI inputs against libmobi v0.10, what memory-safety failures are reproducible, and what can be concluded from controlled sanitizer, corpus and dictionary comparisons?

This document covers only the Honggfuzz workstream. The course-wide comparison against AFL++ and zzuf or Radamsa remains a group deliverable; no teammate results are assumed. The report deliberately omits a general introduction to fuzzing, consistent with the assignment's expectations of a technically informed audience.

1.1 Evidence classes

Class	Definition	Use in this report
A — log-verified	Direct comparison of 17 raw Honggfuzz final summaries with exported TSV.	Campaign counters, durations, coverage and crash events.
B — recorded historical	Previously transcribed in conversation, but primary log unavailable.	Only explicitly tagged HF-02 and HF-03 historical values.
C — direct diagnostics	ASan/UBSan stderr, 12 regression results, Valgrind logs and 5 MOBI samples.	Fault-site confirmation, patch results, SHA-256 and parsed FONT fields.
E — saved analysis	Coverage CSVs, scripts, aggregate report and LCOV comparison text.	Minimization and LLVM comparisons; LCOV and profiles verified in supplement.
D — public sources	Upstream Git commits, release notes, NVD and OSS-Fuzz documentation.	Known issue classification and release context.

Two evidence packages were inspected: a primary 124-file ZIP and a 13-file supplement containing all nine original seeds, two LCOV exports and two merged LLVM profiles. All 17 retained Honggfuzz final summaries match the campaign TSV. Five MOBI evidence samples were hashed, and the original valid FONT seed was independently compared byte-by-byte against the one-byte crash reproducer. Direct LCOV identifier-set comparison confirmed no coverage lost or gained by the minimized corpus. Full corpora and instrumented executables were not provided; no new fuzzing campaigns or 2,462-input coverage replay were run.

2 Target, environment and builds

2.1 System under test

libmobi is an open-source C library for parsing and manipulating Kindle/MOBI electronic-book formats. The tested revision was v0.10 (Git tag aac5b1f578b284b2f886d6d25cde22a33d8534f8). The command-line mobitool executable processed single MOBI inputs and wrote extracted resources to an output directory. The most recent tagged upstream release visible during the source review was v0.12; v0.10 is therefore a historical target rather than the latest upstream version [1, 2].

Target attribute	Value / evidence
Name / language	libmobi / C
Input format	MOBI e-book files
Test version	0.10, older release
Latest visible release at review	0.12 (GitHub releases, accessed 23 Oct 2026)
Driver	mobitool invoked via isolation wrapper
Initial corpus for controlled trials	9 original files; approximately 5.2 MB in prior notes

Target attribute	Value / evidence
Primary platform	Kali Rolling 2026.3, VMware VM, x86_64
Reported resources	4 virtual CPUs; approximately 1.9 GiB RAM and 953 MiB swap
Fuzzer	Honggfuzz; hfuzz-clang compiler wrapper
Compiler	Debian Clang 21.1.8 (13)
OSS-Fuzz public enrollment	No standalone public registration found (23 Oct 2026); see §10.3.

2.2 Verified compiler and linker settings

The two principal v0.10 Autotools builds were configured with identical library settings: --disable-shared --enable-static --with-libxml2=no. The compiler command included the Honggfuzz wrapper and -std=gnu23. The sanitizer difference was documented in the saved Makefiles, not inferred from directory names.

Field	No-ASan build	ASan build
CC	hfuzz-clang -std=gnu23	hfuzz-clang -std=gnu23
CFLAGS	-O1 -g -fno-omit-frame-pointer	-O1 -g -fno-omit-frame-pointer -fsanitize=address
LDFLAGS	(empty)	-fsanitize=address
ASan report functions (nm)	0	34

The non-ASan binary exposed weak ASan compatibility symbols, but zero __asan_report_load/store functions; the ASan binary exposed 34 reporting functions. This supports the separation of configurations. The presence of different Honggfuzz guard counts also indicates that coverage feedback across builds has different instrumentation denominators.

For UBSan with Honggfuzz feedback, a separate CMake build used hfuzz-clang with -O1 -g -fno-omit-frame-pointer -fsanitize=undefined -fno-sanitize-recover=undefined, linking with -fsanitize=undefined. ASan regression builds of v0.11/v0.12 used Clang with -O1 -g -fsanitize=address -fno-omit-frame-pointer and corresponding sanitizer linker flags. A source compatibility include was added during initial build setup; that adaptation is not claimed to be a security fix.

3 Experimental design and measurement

3.1 Execution strategy

The experiment used coverage instrumentation built into the target via Honggfuzz's compiler wrapper. The driver wrapper created isolated extraction directories and invoked mobitool on each fuzzed input. Controlled later campaigns used one worker, a 10-second per-execution timeout, five mutations per run and a 4 MiB maximum input size. A nominal one-hour experiment yielded approximately 3,601 seconds in the logged final summaries. Not every early campaign used the same initial corpus or driver; this distinction is maintained throughout.

The controlled ASan versus non-ASan runs HF-10/HF-11 (61 seconds) and HF-12/HF-13 (3,601 seconds) each began with nine files. The extended HF-09 run instead began with 796 carried-forward corpus files, and HF-14 was a dedicated corpus-minimization operation (not a discovery campaign).

3.2 Metrics and counting rules

- Iterations = total recorded execution iterations of that campaign. This is not a count of unique input files and not necessarily a separately measured total of random mutation operations.
- New units = saved/accepted feedback-distinct inputs or equivalent Honggfuzz reporting metric, not the number of all executed test cases.
- Crashes = Honggfuzz-recorded events; distinct security flaws require manual reproduction, stack-trace clustering and source investigation.
- Timeouts = slow or time-limited runs; they require an independent hang check and do not imply a vulnerability.
- Coverage = Honggfuzz branch-feedback percentage for that instrumented build, not portable source-coverage percentage across arbitrary builds.
- Seed files = count identified for each run by the on-disk audit, which may represent a reused corpus, not necessarily the original nine files.

3.3 Observability and exclusions

HF-05 and HF-06 unintentionally used direct executable invocation with persistent=true and were excluded from controlled performance comparisons. HF-07 contained discarded/incomplete direct-invocation attempts and two directories without a parseable final summary. HF-17 used UBSan without coverage guards (guard count zero): although it executed test cases, it did not provide the intended coverage-guided feedback. The corrected UBSan feedback appeared in HF-18 and HF-19 with 11,912 guards. These experiments remain in the audit for transparency.

4 Complete campaign register (HF-01–HF-21)

Table 1 provides all campaign IDs in order. "—" means that the value was not available in a preserved parseable summary. "Crashes" is a raw fuzzer counter and should not be interpreted as unique findings. This register deliberately separates the seed-file count of each run and flags data-quality exceptions.

ID	Configuration	Seeds	Time s	Iterations	Crash	T/O	HF cov.	Record
HF-01	No ASan · early	9	3883	16,659	0	2	19%	Log
HF-02	ASan · smoke	—	—	—	—	—	—	No summary
HF-03	ASan · long	—	—	—	—	—	—	No summary
HF-04	Timeout triage	652	901	22,269	2	1	19%	Triage only
HF-05	Direct/persistent	879	61	2,805	0	0	18%	Invalid config
HF-06	Direct/persistent	879	—	—	—	—	—	Invalid/no sum.
HF-07	Discarded extended	1128	—	—	—	—	—	No summary
HF-08	Wrapper smoke	796	61	1,679	0	0	18%	Log
HF-09	ASan · 8 h	796	28801	565,277	1	0	22%	Log
HF-10	No ASan · smoke	9	61	768	0	0	17%	Log
HF-11	ASan · smoke	9	61	1,189	0	0	17%	Log
HF-12	No ASan · 1 h	9	3601	41,707	0	1	20%	Log
HF-13	ASan · 1 h	9	3601	84,003	1	0	20%	Log
HF-14	Corpus minimization	1467	990	24,191	0	0	20%	Minimization
HF-15	No dictionary	9	3601	89,267	0	0	20%	Log
HF-16	Dictionary	9	3601	88,734	0	0	20%	Log

ID	Configuration	Seeds	Time s	Iterations	Crash	T/O	HF cov.	Record
HF-17	UBSan · no guards	9	61	2,778	0	0	0%	0 guards
HF-18	UBSan · feedback	9	61	677	0	0	11%	Log
HF-19	UBSan · 1 h	9	3601	36,491	1	0	13%	Log
HF-20	Dictionary repeat	9	3601	68,283	0	0	19%	Log
HF-21	No dict. repeat	9	3601	77,005	0	0	20%	Log

Source: reported 21-row Kali campaign-audit TSV, exported by honggfuzz_campaign_audit.py. HF-02/HF-03 have corpus directories but lack final log summaries; HF-06 has a log without a final summary; HF-07 has no parseable summary in either recorded directory.

Historical-only entries: HF-02 was previously recorded as 1,097 iterations / 61 seconds / zero crashes / 17% feedback; HF-03 as 9,747 iterations / 3,601 seconds / zero crashes / 18% feedback. These figures are not part of Table 1 because the saved log excerpts are absent.

5 Main results and comparison of configurations

5.1 ASan and non-ASan controlled comparison

The most comparable one-hour measurements are HF-12 (no ASan) and HF-13 (ASan), both with nine initial inputs and 3,601 seconds of logged duration. HF-12 recorded 41,707 iterations, zero crashes, one timeout and 20% feedback; HF-13 recorded 84,003 iterations, one crash, zero timeouts and 20% feedback. The 61-second matched smoke tests HF-10/HF-11 showed 768 versus 1,189 iterations, respectively, and both recorded 17% feedback.

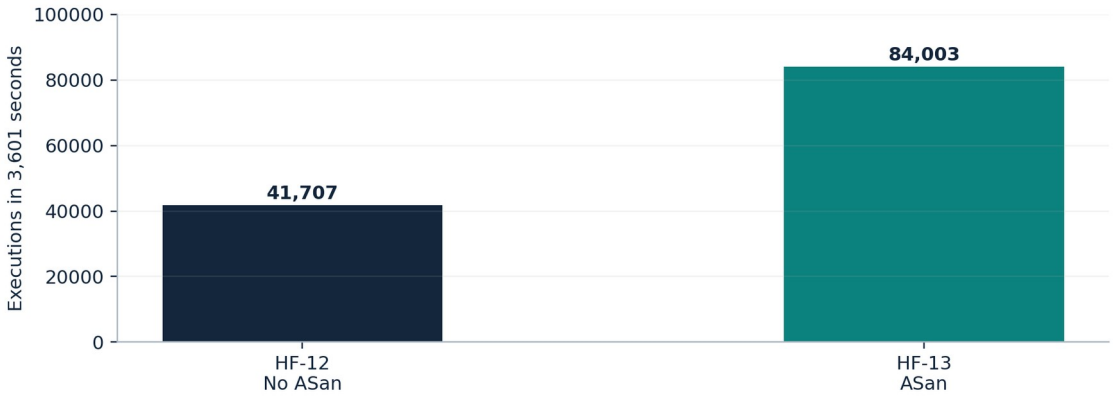


Figure 1. Recorded iterations in the one-hour matched-seed comparison (HF-12 / HF-13).

It would be misleading to conclude that ASan intrinsically improves throughput. The observed difference may reflect build/runtime differences, target behavior, resource constraints or other uncontrolled factors; the underlying cause was not isolated experimentally. Instrumented guard totals differed (8,035 versus 8,103), and the observed maximum resident memory also differed (891 versus 480 MB for HF-12 and HF-13). The valid conclusion is specific to these executions: the ASan campaign executed more iterations and detected a crash that the paired no-ASan run did not record.

5.2 Long campaigns and seed carryover

HF-09 ran for 28,801 seconds and executed 565,277 iterations; it recorded one crash, no timeout, 1,341 new units and 22% reported Honggfuzz coverage. Its initial corpus contained 796 files. The longer duration and larger starting corpus jointly differ from the nine-seed one-hour trials: one cannot attribute the 22% feedback figure solely to an eight-hour runtime. HF-01, an earlier no-ASan campaign, ran 3,883

seconds and recorded 16,659 iterations, 904 new units, two timeouts and 19% feedback. Its specific launch configuration and resource behavior also differ from later controlled trials.

5.3 UBSan and feedback integrity

HF-17, a UBSan smoke test, ran 2,778 iterations in 61 seconds but reported guard_nb=0 and 0% feedback. This was a configuration failure for coverage-guided fuzzing, not evidence that no program logic was reached. A rebuild with hfuzz-clang produced 11,912 guards in HF-18 and HF-19. HF-19 then ran 36,491 iterations in 3,601 seconds, saved one crash and reached 13% reported feedback. Its instrumentation denominator differs from the ASan runs, so 13% and 20% are not directly comparable percentages.

5.4 Crash triage rather than crash counting

HF-04 reported two “crashes” and one timeout in 901 seconds. The saved events were associated with SIGVTALRM/time-limiting behavior and did not reproduce as an independent source-level vulnerability. They are explicitly excluded from the confirmed-bug count. HF-09, HF-13 and HF-19 each generated a different crashing input, but root-cause analysis found the same font-resource decoder fault. Thus the confirmed security finding count remains one.

6 Confirmed vulnerability and crash reproduction

6.1 Failure location and mechanism

The defect occurs in mobi_decode_font_resource in libmobi v0.10, at src/util.c:2543 for the vulnerable build. The parser reads attacker-controlled FONT-resource header fields, including the payload offset, XOR-key length and XOR-key offset. It then forms a pointer into the resource and performs XOR deobfuscation without adequately validating the key pointer’s location. A malformed key offset can point outside the resource buffer and cause an out-of-bounds read, observed as a process crash.

```
const unsigned char *xor_key = buf->data + h.xor_data_off;
...
buf->data[buf->offset++] ^= xor_key[i % h.xor_key_len];
// Fault observed at src/util.c:2543 (libmobi v0.10)
```

The essential missing invariant is that the entire key range must lie inside the valid buffer. Validation must reject an invalid payload offset or an XOR key offset/length pair whose end would exceed buf->maxlen. The hardening must also avoid unchecked integer addition in the validation itself.

6.2 Reproduction evidence

Evidence sample	Origin	Finding
findings/crash-01/crash.mobi	HF-09 (ASan)	Crash reproduced at util.c:2543
findings/crash-02/crash.mobi	HF-13 (ASan)	Same root cause; different crafted offset
findings/ubsan-crash-01/crash.mobi	HF-19 (UBSan build)	Same source fault; separately saved sample
findings/crash-02/one-byte-reproducer.mobi	Single-byte manual reduction	Same fault using an otherwise valid original seed
findings/crash-02/record-minimized.mobi	Structure-aware minimized sample	Same fault in a substantially smaller file

A crash-inducing file is not by itself proof of a new vulnerability or code execution. Here the failure was linked to a concrete unchecked read, independently confirmed by Memcheck and matched to subsequent upstream fixes. No exploitability or remote attack vector was demonstrated by the experiment.

6.3 One-byte trigger

The original valid seed, sample-obfuscated-fonts.mobi, and the archived one-byte crash reproducer are both 84,412 bytes. An independent byte-by-byte comparison confirms exactly one modification, at zero-based offset 15,378 (one-based position 15,379): 0x00 became 0xFF. This changes the XOR key offset from 0x00000018 (24) to 0xFF000018 (4,278,190,104). The reproducer contains the FONT marker, flags=3, data offset=44 and key length=20; its saved ASan stderr shows a read-triggered SEGV at src/util.c:2543. SHA-256 of the valid seed: 34fc67043eeeaa6563481d6ea2fea1b5349cb73e459a1f10e3aa09186f56302e.

Property	Recorded value
Original seed size	84,412 bytes
Edited byte position	15,378 (zero-based)
Original -> modified byte	0x00 -> 0xFF
Original key offset	0x00000018 (24)
Modified key offset	0xFF000018 (4,278,190,104)
Difference from original seed	Confirmed independently by exact seed-to-reproducer byte comparison

7 Crash and corpus minimization

7.1 Structure-aware crash minimization

An initial naive byte-deletion strategy performed 350 trials without finding a smaller reproducible file. A later MOBI/PalmDB-aware reducer truncated the affected FONT record to 45 bytes and adjusted following record offsets. This reduced the full MOBI file from 84,412 bytes to 15,531 bytes: 68,881 bytes removed (81.60%). A 44-byte FONT record did not reproduce the same crash in the boundary test. “Minimum” here means smallest demonstrated using that specific structure-aware reduction, not a formal global minimum over all possible inputs.

Metric	Before	After	Change
Total reproducer size	84,412 B	15,531 B	-81.60%
FONT record size	68,926 B	45 B	-68,881 B
Crash reproduction	Yes	Yes	Preserved

7.2 Honggfuzz corpus minimization

HF-14 used Honggfuzz corpus minimization (-M) on the 1,467-file HF-13 corpus. It logged 24,191 analysis executions over 990 seconds. These iterations are minimization evaluations and are not counted as a normal discovery-run mutation total. The minimized corpus retained 995 files and 276,162,698 bytes, compared with 1,467 files and 480,881,076 bytes originally.



Figure 2. Corpus reduction in file count and total stored bytes (HF-13 -> HF-14).

7.3 Independent source-coverage equivalence

Saved coverage-collection logs and per-input CSVs account for 1,467 original and 995 minimized inputs; the log reports profile creation for all 2,462 executions. In the supplemental archive, both merged .profdata binaries are present and parse correctly. The two LCOV exports were independently parsed: each contains records for 28 source files, with identical aggregate counts (FNH/FNF 276/566, LH/LF 5,710/14,913, BRH/BRF 2,424/8,112). Those aggregate source-level totals are consistent with the previously reported 48.76% function, 38.29% line and 29.88% branch coverage. Region coverage of 35.77% remains taken from the saved llvm-cov report, not recomputed from LCOV.

LLVM reporting metric	Original	Minimized	Result
Region coverage	35.77%	35.77%	Identical
Function coverage	48.76%	48.76%	Identical
Line coverage	38.29%	38.29%	Identical
Branch coverage	29.88%	29.88%	Identical
LCOV line IDs covered	5,921	5,921	No lost/gained IDs
LCOV branch IDs covered	2,395	2,395	No lost/gained IDs
LCOV function IDs covered	276	276	No lost/gained IDs

Independent covered-identifier set comparison of the two supplied LCOV files, keyed by source file and item identity, yields exactly 5,921 covered lines, 2,395 covered branch identifiers and 276 covered functions in both corpora, with zero lost and zero gained identifiers. The identifier-set counts are not interchangeable with the aggregated llvm-cov/LCOV line and branch summary totals (for example, LH=5,710). Equality of these source-level coverage sets does not establish equivalent Honggfuzz-internal feedback or functional behavior.

8 Sanitizer cross-validation and custom fix

8.1 What each diagnostic tool established

Tool / build	Direct observation	Inference limit
ASan v0.10	Saved stderr: AddressSanitizer: SEGV, caused by READ at util.c:2543.	SEGV trace is not a structured ASan heap-buffer-overflow report.
UBSan v0.10	Saved stderr: UndefinedBehaviorSanitizer: SEGV, caused by READ at util.c:2543.	Signal trace does not establish a distinct UBSan rule violation.
Valgrind Memcheck	Invalid read of size 1 at mobi_decode_font_resource, util.c:2543.	Memory error does not by itself establish code execution.
ASan v0.11 and v0.12	The selected reproducers no longer triggered the identified sanitizer fault.	A nonzero parser exit may still reflect rejection of malformed input.

8.2 Patch design

The custom patch validates the data offset and XOR key range before pointer dereference. Its key-range condition uses the subtraction-safe form below, preventing wraparound that would be possible with unchecked addition of two untrusted lengths. The patch returns MOBI_DATA_CORRUPT for a detected invalid font-data configuration.

```
if (h.data_offset > buf->maxlen ||
    h.xor_key_len > buf->maxlen ||
    h.xor_data_off > buf->maxlen - h.xor_key_len) {
    debug_print("%s\n", "Invalid obfuscated font data offsets");
    mobi_buffer_free(buf);
    return MOBI_DATA_CORRUPT;
}
```

8.3 ASan regression matrix

Six inputs were run once on the original v0.10 ASan executable and once on a separately built custom-patched ASan executable, for 12 reported trials. The valid seed completed successfully on both. All five malicious test cases reproduced the same fault on the vulnerable binary; none reproduced that sanitizer fault on the patched binary. This is a bounded regression test, not a general proof of memory safety.

Input case	Original ASan v0.10	Custom-patched ASan v0.10
Valid original seed	Exit 0	Exit 0
One-byte reproducer	Fault / SIGABRT	Exit 0; no known fault
Record-minimized reproducer	Fault / SIGABRT	Exit 0; no known fault
HF-19 saved crash	Fault / SIGABRT	Exit 0; no known fault
HF-09 saved crash	Fault / SIGABRT	Exit 1, parser rejection; no sanitizer fault
HF-13 saved crash	Fault / SIGABRT	Exit 1, parser rejection; no sanitizer fault

8.4 Valgrind regression matrix

Memcheck compared vulnerable and patched non-sanitized builds on the valid seed, one-byte reproducer and size-minimized reproducer (six trials). The valid seed produced zero memory errors in both builds. The two malformed reproducers produced an “Invalid read of size 1” and exited with the configured error

code 99 on the vulnerable build; patched executions reported zero Memcheck errors and exit 0. The recorded runs reported no definitely, indirectly or possibly lost bytes in these reproductions; the vulnerable runs do list some "still reachable" allocations, which is a separate category and not proof of a memory leak.

9 Dictionary-based fuzzing experiment

9.1 Experimental pairing

A custom dictionary of 15 MOBI-specific tokens was tested against a baseline without a dictionary. Both conditions were repeated twice, beginning with nine source files and running for 3,601 logged seconds each. Runs were performed serially and the second pair reversed order (dictionary first, baseline second) to reduce a simple fixed-order explanation. The exact mutation random seeds were not captured in the supplied summary, so the repeats are observational rather than reproducible fixed-seed trials.

ID	Dictionary	Iterations	New units	Feedback	Crashes
HF-15	No	89,267	1,418	20%	0
HF-16	Yes	88,734	1,407	20%	0
HF-20	Yes	68,283	1,309	19%	0
HF-21	No	77,005	1,275	20%	0

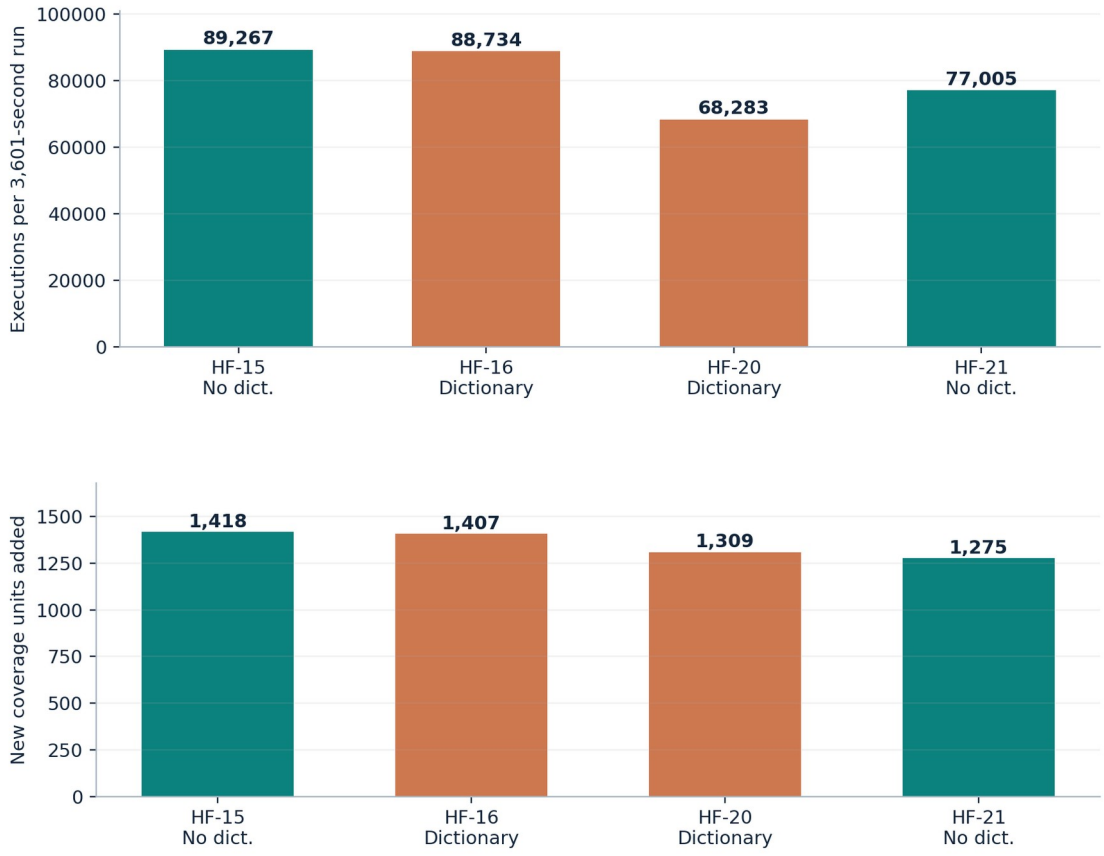


Figure 3. Separate observed throughput and new-unit outcomes in repeated dictionary trials.

9.2 Descriptive statistics and interpretation

Aggregate (2 runs/condition)	No dictionary	Dictionary
Mean iterations	83,136	78,508.5
Mean new units	1,346.5	1,358
Mean reported coverage*	20.0%	19.5%
Total recorded crashes	0	0

*Coverage values were rounded by Honggfuzz and the means should be interpreted only descriptively. Across the two pairs, dictionary-enabled tests averaged about 5.57% fewer executions and 0.85% more new units than baseline. However, no reproducible coverage improvement or new defect emerged. Two trials per condition and considerable throughput variability preclude a statistically meaningful superiority claim.

10 Upstream history, CVEs and OSS-Fuzz status

10.1 Independent upstream fix confirmation

Local history review identified the initial security commit 811bac3d32ec06d86cb131008d30dfa4b8ca1336, dated 23 April 2022, with the message “Fix checking boundary of deobfuscation key which could cause buffer over-read with corrupt data” [3]. A second commit, 6269ea673cb583f7a2aab92dd0f8113274742b7d, dated 17 May 2022, changed the range check to avoid potential integer overflow [4]. These commits were found after fetching the full upstream history while leaving the working tree at the vulnerable v0.10 tag.

The v0.11 release is explicitly described upstream as a bug-fix/security release; v0.12 is the newest tagged release shown at the review date [2]. The exact font-decoder crash did not reproduce in local ASan replays against either tested later version. Therefore, the independently observed failure should be classified as a previously fixed historical issue, not a zero-day.

10.2 CVE caution

libmobi has known CVEs, and upstream v0.11 release notes list seven identifiers: CVE-2022-1533, CVE-2022-1534, CVE-2022-1907, CVE-2022-1908, CVE-2022-1987, CVE-2022-2279 and CVE-2022-29788 [2]. Public NVD entries for at least CVE-2022-1533 and CVE-2022-1908 reference other individual code-fix commits [5, 6]. No reviewed evidence unambiguously maps the font XOR offset defect to a specific CVE identifier. The finding is therefore called “known upstream-fixed issue; exact CVE assignment unconfirmed.”

10.3 OSS-Fuzz membership

As of 23 October 2026, no standalone public OSS-Fuzz registration for libmobi was identified. The official OSS-Fuzz integration guide specifies a dedicated projects/<project_name>/project.yaml entry for each enrolled project. The expected public configuration paths projects/libmobi/project.yaml and projects/mobi/project.yaml returned HTTP 404. The public Fuzz Introspector project overview also contained no entry matching 'libmobi' or 'mobi'; however, that overview explicitly covers only a subset of enrolled projects. These checks support the classification 'No publicly listed standalone OSS-Fuzz integration found' rather than an absolute claim that the library is never fuzzed by OSS-Fuzz indirectly or under another project's name [7, 11].

11 Threats to validity and methodological reflection

Differing starting corpora

HF-09 began with 796 files, whereas the most comparable one-hour ASan/no-ASan campaigns began with nine. Coverage or defect yield cannot be attributed to elapsed time alone.

Incomplete primary logs

Four campaigns lack parseable final summaries. Any previously recorded numerical values for HF-02/HF-03 must remain labeled historical and should not silently enter the log-derived table.

Run configuration errors

Accidental persistent mode contaminated HF-05/HF-06 and probably discarded HF-07 configurations. HF-17 was not coverage-guided because its binary exposed zero guards.

Coverage denominator mismatch

Honggfuzz guard totals of 8,035, 8,103 and 11,912 indicate different instrumentations. Percentages across builds are not a common absolute coverage scale.

Sanitizer performance

ASan ran faster in the specific controlled comparisons; that observation does not isolate sanitizer overhead because the execution environment and code paths may differ.

Limited statistical replication

Only two repeated dictionary runs per setting, without captured random-seed control or performance confidence intervals, are insufficient for statistical significance.

Crash duplication and impact

Independent crash files can share one root cause. Timeout signals were triaged out. No demonstrated arbitrary code execution, confidentiality leak or remote exploitation should be claimed.

Patch scope

Regression success on selected malformed inputs does not prove that all invalid font headers or unrelated parsers are now memory safe.

11.1 Practical engineering lessons

The most consequential setup errors were not sophisticated: the wrong driver caused unintended persistent mode, and one UBSan build lacked the coverage feedback necessary for the planned guided test. Explicit verification of the executable, wrapper, guard count and control-run behavior was more important than increasing nominal fuzzing runtime. Memory pressure on the small Kali VM and the very large saved corpora also favored careful disk/corpus management and smaller, comparable experiments.

12 Conclusions and assignment hand-off

Honggfuzz uncovered one previously fixed, reproducible out-of-bounds read in libmobi v0.10. The root cause was an unchecked FONT-resource XOR key range in mobi_decode_font_resource. Multiple saved crashing inputs converged on the same code location; a one-byte edit isolated the offending field. AddressSanitizer-instrumented replays, UBSan-build signal reproduction and Valgrind Memcheck supported the diagnosis. Regression results supported a subtraction-safe range-check patch, consistent with the two relevant upstream commits.

A structure-aware reducer cut the crashing file by 81.60%, and a corpus-minimization pass removed 42.57% of corpus bytes while maintaining the measured LLVM source-level coverage sets. Two dictionary trials per setting did not identify a stable coverage advantage. The operational outcome is a reproducible and well-evidenced Honggfuzz workstream, with methodological caveats and historical failures preserved rather than erased.

REMAINING GROUP-LEVEL RESPONSIBILITY

The assignment additionally requires AFL++ and zzuf or Radamsa, plus a cross-fuzzer performance comparison and group reflection. These cannot be completed from the Honggfuzz material alone and are intentionally not fabricated in this standalone report.

Appendix A Detailed audit and measurement provenance

A.1 Raw campaign metrics beyond the main register

The following auxiliary values are transcribed from the saved TSV where available. They support environment and instrument comparisons but should not be read as separately measured generated-mutation totals.

ID	New units	Guards	Peak RSS (MB)	Corpus files	Saved crash files
HF-01	904	8035	1544	958	0
HF-02	—	—	—	392	0
HF-03	—	—	—	652	0
HF-04	366	8103	245	879	2
HF-05	0	8103	90	243	0
HF-06	—	—	—	796	0
HF-07	—	—	—	0	0
HF-08	0	8103	82	160	0
HF-09	1341	8103	593	1978	1
HF-10	284	8035	918	339	0
HF-11	367	8103	137	421	0
HF-12	1200	8035	891	1253	0
HF-13	1414	8103	480	1467	1
HF-14	0	8103	282	—	0
HF-15	1418	8103	409	1469	0
HF-16	1407	8103	399	1462	0
HF-17	0	0	46	1	0
HF-18	290	11912	1410	344	0
HF-19	1128	11912	1537	1177	1
HF-20	1309	8103	456	1362	0
HF-21	1275	8103	385	1330	0

For HF-14, the audit script reported the generic corpus/ path as unavailable even though the actual minimized output was checked independently under runs/HF-14-corpus-minimization/output. HF-17 produced an unusual empty/one-file output during the non-guided smoke test, and this value is not used as a valid coverage corpus.

A.2 Representative controlled commands

The following is a representative invocation drawn from the separately recorded HF-20/HF-21 setup, not a literal reconstruction of every campaign. Relative paths are relative to ~/Desktop/Fuzzing_Project. Execution flags may differ in earlier campaigns.

```
ASAN_OPTIONS=detect_leaks=0:abort_on_error=1 \
./honggfuzz/honggfuzz \
  -i runs/HF-20-dict-repeat/input \
  -o runs/HF-20-dict-repeat/corpus \
  -W runs/HF-20-dict-repeat/work \
  -l runs/HF-20-dict-repeat/honggfuzz.log \
  -v -n 1 -t 10 -r 5 -F 4194304 \
  -w "$PWD/dictionaries/mobi.dict" \
  --run_time 3600 \
  -- ./run-comparison-asan.sh ___FILE___
```

HF-21 reused the same family of flags and the wrapper without the -w dictionary option. Independent run directories and fresh copies of the nine seeds were used.

Appendix B Reproducibility and evidence manifest

The operator completed a 16/16 on-machine path-presence audit. The first ZIP contained 124 evidence files; the follow-up supplement provided nine original MOBI seeds and four source-coverage artifacts (two LCOV exports and two merged .profdata files). Five failure/reproducer samples were previously hashed (Appendix C), and the new seed archive enabled independent hash and byte-level comparison. Full fuzzing corpora and compiled executables remain unavailable in the submitted archives.

Evidence type	Relative path
Campaign audit	reporting/honggfuzz/campaigns-HF01-HF21.tsv
Audit summary	reporting/honggfuzz/campaign-audit-summary.txt
Compiler evidence	reporting/honggfuzz/verified-build-configuration.txt
Original nine-file seed corpus (supplement)	seeds-original/
HF-09 crash	findings/crash-01/crash.mobi
HF-13 crash	findings/crash-02/crash.mobi
HF-19 crash	findings/ubsan-crash-01/crash.mobi
One-byte reproducer	findings/crash-02/one-byte-reproducer.mobi
Size-minimized reproducer	findings/crash-02/record-minimized.mobi
Custom patch	findings/crash-02/custom-security-fix.patch
ASan regression matrix	findings/customfix-regression/summary.tsv
Memcheck analysis	reporting/honggfuzz/valgrind-analysis.txt
Exact LLVM coverage sets	reporting/honggfuzz/coverage/exact-coverage-comparison.txt

Evidence type	Relative path
Coverage comparison	reporting/honggfuzz/coverage-final-analysis.txt
Dictionary analysis	reporting/honggfuzz/dictionary-repeated-analysis.txt
Upstream history	reporting/honggfuzz/vulnerability-history.txt
Upstream initial diff	reporting/honggfuzz/upstream-fixes/initial-fix.patch
Upstream overflow-hardening diff	reporting/honggfuzz/upstream-fixes/overflow-fix.patch
Source LCOV (original)	reporting/honggfuzz/coverage/original.lcov
Source LCOV (minimized)	reporting/honggfuzz/coverage/minimized.lcov
Merged LLVM profiles	reporting/honggfuzz/coverage/{original,minimized}/merged.profdata

B.1 Report limitations before final submission

- The evidence ZIP includes 17 final-summary logs and selected ASan, UBSan and Valgrind stderr; retain the ZIP and hashes with the hand-off package.
- Attach both archived evidence packages: the primary logs, crash artifacts and patch records, plus the supplemental nine valid seeds and LLVM LCOV/profile files. Include the 21-run TSV and compiler configuration for review.
- Obtain independently reported AFL++ and zzuf/Radamsa results from teammates and add a separate group-level comparison; do not merge them into the Honggfuzz experiment counts.
- Avoid archiving all large 100s-of-MB corpora as default evidence; preserve path references or a reproducible minimal subset and hashes if the course requires it.

Appendix C Primary-evidence verification

C.1 Archive integrity and log consistency

The supplied evidence ZIP contained 124 files (approximately 6.86 MiB uncompressed). The full final-summary lines in the 17 preserved Honggfuzz logs were matched field-by-field against the campaign TSV: iterations, duration, crashes, timeouts, new units, guards, reported coverage and peak RSS. No numeric discrepancies were found. HF-02, HF-03, HF-06 and HF-07 have no parseable final summary; historic values for HF-02/03 are not log-verified.

C.2 Hash-identified reproduction inputs

SHA-256 was computed directly from the five MOBI files in the uploaded ZIP. Truncated hashes below are identifiers; full hashes are retained in the accompanying evidence-verification audit.

Archive input	Bytes	SHA-256 prefix (16 hex)
HF-09 / crash-01	84,421	540eb6dc8ba4b8a6
HF-13 / crash-02	84,418	79921cc5ea2c1a0
HF-19 / ubsan-crash-01	84,412	5671ff8d36458253
One-byte reproducer	84,412	3153f56016c369e8
Structure-aware minimized	15,531	724100e07b47e281

The one-byte reproducer contains the FONT marker at byte 15,358, data offset 44, XOR key length 20 and invalid key offset 0xFF000018 in the four bytes beginning at file offset 15,378. The supplemental original

valid seed matches the reproducer in every byte except zero-based offset 15,378 (0x00 -> 0xFF); both files have length 84,412 bytes. This directly verifies the claimed one-byte reproducer.

C.3 Raw diagnostics and regression checks

The archived one-byte ASan log records "AddressSanitizer: SEGV" caused by a READ at src/util.c:2543. The independent UBSan-build log also records a read-triggered SEGV at the same code site; no different typed UBSan rule violation was shown. All five malicious cases in the 12-row regression TSV generated that location on the vulnerable ASan build and none generated sanitizer findings in the patched build. Four patched cases exited 0, two rejected malformed input with exit 1. Valgrind produced one "Invalid read of size 1" error context on each of two vulnerable reproductions and zero memory-error contexts in all three patched tests.

C.4 Coverage evidence and remaining reproducibility gap

The archived execution CSVs contain 1,467 original executions (498 exit 0; 969 exit 1) and 995 minimized executions (347 exit 0; 648 exit 1). The collection log reports a profile generated for each case. A supplemental archive now supplies both LCOV exports and both merged .profdata files. A new direct comparison of the LCOV identifiers confirms zero lost or gained covered lines, branches or functions. The original seeds are also present, but the instrumented executable and full fuzzing corpora were not supplied; the 2,462 input executions were not independently rerun.

Appendix D Original seeds and source-coverage verification

The second evidence package contains the entire nine-file seed set (5,392,226 bytes; 5.14 MiB), both original/minimized LCOV exports and both LLVM merged profiling files [10]. The LCOV exports cover 28 source files each. Both .profdata files were validated using llvm-profdata show; instrumented libmobi executables were not supplied.

D.1 Byte-level reproducer check

Comparing seeds-original/sample-obfuscated-fonts.mobi with findings/crash-02/one-byte-reproducer.mobi finds exactly one differing byte in 84,412 positions, at zero-based position 15,378: 0x00 -> 0xFF. The SHA-256 of the valid original is 34fc67043eeeaa6563481d6ea2fea1b5349cb73e459a1f10e3aa09186f56302e. This directly confirms the stated one-byte trigger, independently of the earlier experiment notes.

D.2 Exact coverage-preservation check

The two LCOV files were independently parsed by source file. Their nonzero hit-count line, branch and function identifier sets were compared directly; all sets are identical (zero lost, zero gained identifiers). Reported LF/LH and BRF/BRH summary fields also agree exactly, but must not be conflated with the distinct hit-identifier cardinalities.

Covered identifiers	Original	Minimized	Lost	Gained
Lines	5,921	5,921	0	0
Branches	2,395	2,395	0	0
Functions	276	276	0	0

D.3 Verification boundary

This adds an independent verification of archived coverage exports and the original seed bytes. It does not constitute an independent replay of all 2,462 corpus cases, an end-to-end recompilation, a proof of identical Honggfuzz feedback, or a claim that the record-minimized crash is globally minimal. The concise validation record is provided separately as Honggfuzz_Supplement_Verification.md.

References

[1] bfabiszewski, “libmobi,” project repository. https://github.com/bfabiszewski/libmobi

[2] bfabiszewski, “libmobi releases,” especially versions 0.11 and 0.12, accessed 23 October 2026. https://github.com/bfabiszewski/libmobi/releases

[3] bfabiszewski, “Fix checking boundary of deobfuscation key,” commit 811bac3, 23 April 2022. https://github.com/bfabiszewski/libmobi/commit/811bac3d32ec06d86cb131008d30dfa4b8ca1336

[4] bfabiszewski, “Fix potential integer overflow with corrupt data,” commit 6269ea6, 17 May 2022. https://github.com/bfabiszewski/libmobi/commit/6269ea673cb583f7a2aab92dd0f8113274742b7d

[5] NIST NVD, “CVE-2022-1533,” accessed 23 October 2026. https://nvd.nist.gov/vuln/detail/CVE-2022-1533

[6] NIST NVD, “CVE-2022-1908,” accessed 23 October 2026. https://nvd.nist.gov/vuln/detail/CVE-2022-1908

[7] Google OSS-Fuzz, project repository and new-project documentation, accessed 23 October 2026. https://github.com/google/oss-fuzz

[8] Software Security course, “Fuzzing Group Project,” assignment text provided with the project. Course material (provided file; no public URL)

[9] Local Honggfuzz original experiment logs, crash reproducers, sanitizer/regression records and audit TSV. User-provided 124-file ZIP evidence package, October 2026

[10] Supplemental original nine-file seed corpus and LLVM coverage exports/profiles, provided as honggfuzz_supplement.zip, October 2026. Independently parsed and compared in Honggfuzz_Supplement_Verification.md.

[11] Google OSS-Fuzz, “Setting up a new project” (project.yaml structure), and Fuzz Introspector project overview, accessed 23 October 2026. https://google.github.io/oss-fuzz/getting-started/new-project-guide/ ; https://introspector.oss-fuzz.com/projects-overview